

Adaptive Recurrent Message Passing for Test Time Computing on Graphs

Junshu Sun^{1,2} Wanxing Chang³ Qingming Huang² Shuhui Wang¹

Abstract

Pre-trained foundation models have demonstrated remarkable success in many domains, enabling a unified backbone to generalize across diverse downstream tasks. However, extending this paradigm to graph learning remains challenging due to the intrinsic mismatch between graph data and fixed architectural designs. In this work, we show that this limitation can be overcome via recurrent graph models. To achieve this, we conduct a systematic theoretical analysis, rigorously deriving step dependence as a necessary and sufficient condition for an adaptively convergent recurrent process. Building on this foundation, we propose AdaR, an **Adaptive Recurrent** graph model, empowering flexible test-time computing on various downstream tasks without changing model parameters. To enable adaptive inference, AdaR explicitly encodes normalized step information and representation–target relations into the recurrent updates. To ensure convergence of the recurrent process, AdaR employs gradient-based supervision signals that guide representation updates throughout the recurrence. Empirical results demonstrate that AdaR consistently outperforms strong baselines in both inductive and transductive settings.

1. Introduction

Pre-trained foundation models have achieved remarkable success in many domains, such as natural language understanding (OpenAI et al., 2024) and image processing (Kirillov et al., 2023). This success highlights the potential of a unified paradigm, where a fixed model architecture with a single set of pre-trained parameters can be used for diverse

¹State Key Laboratory of AI Safety, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
²University of Chinese Academy of Sciences, Beijing, China
³DAMO Academy, Alibaba Group, Hangzhou, China. Correspondence to: Shuhui Wang <wangshuhui@ict.ac.cn>.

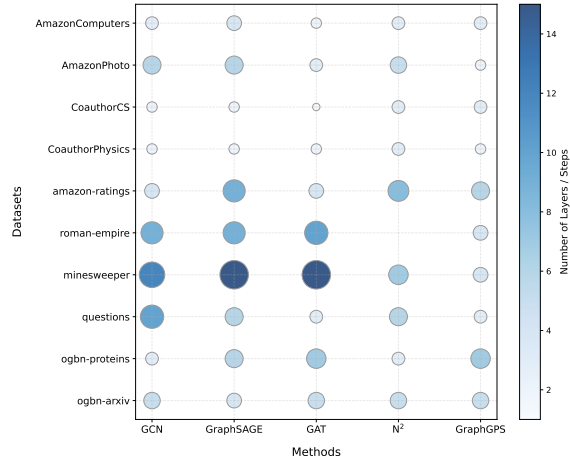


Figure 1. Diversity of Optimal Layer Configurations Across Datasets. Graph models exhibit varying optimal numbers of layers or steps across different datasets. This indicates that effective feature extraction requires the models to adjust their receptive field based on specific datasets. In consequence, pre-trained graph models with a fixed number of layers lack the flexibility to be adapted to diverse datasets.

downstream tasks. However, this paradigm has yet to be validated in graph learning, primarily due to the mismatch between graph data characteristics and pre-defined model architectures.

Specifically, graph data exhibit diverse receptive field requirements, whereas pre-defined backbone architectures impose a fixed receptive field¹. From the data perspective, real-world graphs vary significantly in scale, requiring models to adapt their receptive fields to extract both local neighborhood information and global graph patterns (Fig. 1²). From the architectural perspective, graph neural networks (GNNs) expand their receptive fields via layer stacking, where multi-hop feature extraction is tightly coupled with the number of layers (Kipf & Welling, 2017; Sun et al., 2023a). As a result, pre-trained GNNs with fixed architectures can not adjust their receptive field without altering the model parameters. This mismatch between the adaptive receptive field requirements of graph data and the rigid design of pre-defined architectures poses a fundamental obstacle to

¹Reachable nodes via message passing on the input graph.

²Results are averaged on configurations with the same number of layers to exclude variations caused by other hyperparameters.

realizing one-for-all pre-training on graphs.

To address this problem, previous methods introduce additional adjustments to adapt pure graph models to downstream tasks, either at the data level through graph prompting (Sun et al., 2023b; 2022; Fang et al., 2023) or at the model level via fine-tuning (Gui et al., 2024; Li et al., 2024a). Other methods (Chen et al., 2024a; Gao et al., 2024) further incorporate external knowledge by leveraging language models (LMs) to enhance task adaptability. Nevertheless, it remains an open problem whether a pre-defined graph backbone can be directly adapted to downstream tasks without any additional architectural or data modifications.

In this paper, we answer this question affirmatively through recurrent graph models, which can adjust their receptive fields by computing with different numbers of recurrent steps during test time. We present a systematic study of recurrent graph models, establishing step dependence as a necessary and sufficient condition for adaptive inference under different recurrent step budgets. Based on this theoretical foundation, we propose an **Adaptive Recurrent** graph model, termed **AdaR**, enabling flexible test-time computing on graphs with varying receptive field requirements without altering the model architecture.

To enable an adaptive recurrent process, AdaR explicitly depends on the current *step information*, the current graph node representation, and the *representation-target relations*. Among these, the *step dependence* ensures convergent approximation towards the targets based on our theoretical analysis. By explicitly encoding normalized step information into the recurrent updates, the model is aware of its recurrent progress and avoids out-of-distribution behavior on unseen budgets for the number of recurrent steps. The *relation dependence* enables adaptive inference based on specific inputs and targets. AdaR dynamically computes relations between the current representations and their targets, adjusting the representation updates accordingly. During the training stage, AdaR employs gradient-based supervision signals that guide representation updates throughout the recurrence. Empirically, we demonstrate that AdaR can be trained with a small iteration budget while being evaluated with substantially larger budgets, consistently outperforming baselines on both inductive and transductive tasks. Our contributions can be summarized as follows

- We present a theoretical analysis of recurrent graph models, deriving a general formulation and a necessary and sufficient condition for an adaptively convergent process.
- We design AdaR, an adaptive recurrent graph model, enabling flexible test-time computing on various downstream tasks without changing model parameters.
- We demonstrate the superiority of AdaR on transfer

tasks, outperforming baselines in both inductive and transductive settings.

2. Related Work

2.1. Adaptation with Pure Graph Models

Fine-tuning for model adjustment. A direct solution for adapting graph backbones to diverse downstream tasks is to fine-tune pre-trained models. Primary methods (Qiu et al., 2020; Hu et al., 2020) fine-tune the entire set of model parameters. However, full fine-tuning is resource-intensive and tends to overfit when downstream tasks provide only limited labels. To overcome these limitations, parameter-efficient fine-tuning (PEFT) methods have been proposed for graph models. AdapterGNN (2024a) provides a comprehensive comparison of traditional PEFT techniques in the graph learning domain and introduces dual adapter modules specifically designed for GNN architectures. G-Adapter (2024) further extends PEFT to graph transformers (Ying et al., 2021; Sun et al., 2025a), employing graph convolution operators to inject structural information into the tuning process.

Prompting for data adjustment. Despite their efficiency, the effectiveness of PEFT methods does not always scale favorably with increasing model size (Yu et al., 2024a). Consequently, recent studies have shifted the focus from adjusting model parameters to adapting the input data, aiming to bridge the gap between pre-training and inference. In particular, graph prompt learning proposes to adapt node features or graph structures through task-specific prompts (Sun et al., 2023b; Wang et al., 2025b). For feature-level prompts, auxiliary embeddings are introduced either to unify input node features (Sun et al., 2022; 2023b; Fang et al., 2023) or to refine intermediate layer representations (Liu et al., 2023b; Yu et al., 2024b). For structure-level prompts, substructures such as subgraphs or pseudo nodes are learned to augment the original graph topology (Sun et al., 2023b; Tan et al., 2023; Zhang et al., 2023; Huang et al., 2023). For more discussion on the pseudo nodes in adaptive graph models, please refer to Appendix B.

2.2. Adaptation with Language Models

Language models exhibit strong generalizability across diverse downstream tasks (OpenAI et al., 2024). Leveraging this capability, researchers have explored three ways to incorporate LMs in graph learning: as primary backbones, as knowledge providers for graph models, or as collaborators alongside graph models. To construct backbones with LMs, graphs are converted into sequential data through textual descriptions (Li et al., 2024b) or structure-aware node traversals (Chen et al., 2024a; Gao et al., 2024). Beyond serving as backbones, LMs can also enhance graph models

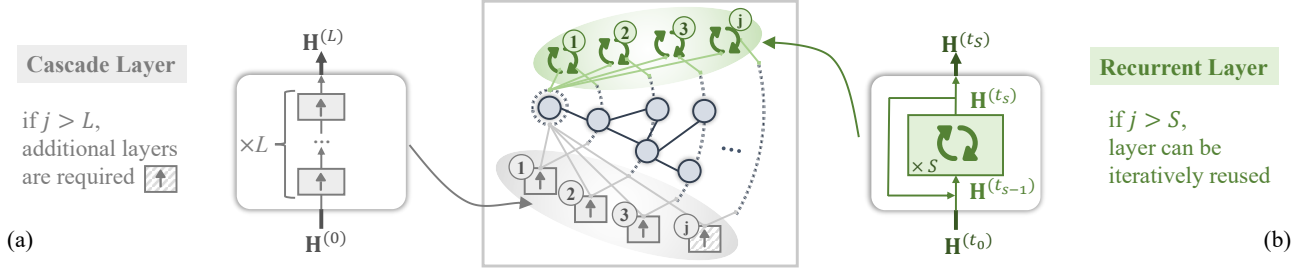


Figure 2. **Comparison Between Traditional and Recurrent Graph Model.** (a) Traditional models stack multiple cascade layers. (b) In contrast, recurrent models employ a recurrent layer, which can be iteratively reused to adapt to different receptive field requirements.

for downstream tasks. Specifically, some methods transfer external knowledge from LMs to graph models via strategies such as knowledge distillation (Xia et al., 2024; Pan et al., 2024) and representation alignment (Zhu et al., 2025). Other methods integrate LMs and graph models, combining topological features captured by graph models with the strong generalizability of LMs (Tian et al., 2024; Zhang et al., 2024; Tang et al., 2024; Huang et al., 2024; Kong et al., 2024; Hu et al., 2024).

2.3. Recurrent Graph Models

Recurrent graph models have demonstrated strong performance in various real-world applications (Selsam et al., 2018; Bresson & Laurent, 2018). Scarselli et al. (2009) first introduced recurrent architectures to graph learning, where model parameters are constrained to satisfy contraction conditions, ensuring convergence to a fixed point via iterative updates. Following the recurrent paradigm, subsequent works relax the contraction constraints and integrate local message passing with gating mechanisms during the recurrent process (Li et al., 2017; Ruiz et al., 2020). More recently, N² extends recurrent models to global message passing by enabling dynamic interactions between graph nodes and learnable pseudo nodes (Sun et al., 2024).

Despite their effectiveness, both gated methods and N² assume the same fixed iteration budget for training and testing, which introduces two key limitations. First, when the optimal number of iterations is large, training incurs prohibitive memory overhead. Second, once trained, these models cannot compute with different iteration budgets, failing to satisfy the diverse receptive field requirements of downstream tasks. To empower variable iteration budgets for flexible test-time computing, we theoretically show that the model must explicitly incorporate step information, and propose AdaR to empirically validate this principle.

3. Method

Local message passing expands the node receptive field by limited hops per step. As a result, capturing broader

graph contexts requires multiple rounds of message passing. Traditional graph models achieve this by stacking multiple cascade layers, each with its own parameters. This stacking strategy enlarges the receptive field but also increases parameter count and changes the architecture. To avoid these drawbacks, a recurrent graph model is necessary, where a single message passing layer is iteratively reused as the recurrent layer to expand the receptive field adaptively (Fig. 2). In this section, we first present a general formulation of recurrent graph models, then derive the condition required for ideal recurrent behavior, and finally implement the principle in a novel recurrent model AdaR.

3.1. Problem Setup

Notations. Given an input graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Let $\mathcal{V} = \{v_1, \dots, v_n\}$ denote the set of nodes with $|\mathcal{V}| = n$, and $\mathcal{E} = \{e_{i,j} \mid v_j \in \mathcal{N}(v_i)\}$ denote the set of edges with $|\mathcal{E}| = m$, where $\mathcal{N}(v)$ is the set of one-hop neighbors of node v . Each node $v \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_v \in \mathbb{R}^{d_{in}}$, where d_{in} denotes the feature dimension. The node feature matrix is then defined as $\mathbf{X} = (\mathbf{x}_{v_1}, \dots, \mathbf{x}_{v_n})^T \in \mathbb{R}^{n \times d_{in}}$. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ denote the adjacency matrix of $\mathcal{G}$, where $\mathbf{A}_{i,j} = 1$ if $e_{i,j} \in \mathcal{E}$ and 0 otherwise.

Graph Learning Tasks. Let c denote the number of labels or attributes in classification or regression. Each label or attribute is associated with a descriptive embedding, which is either provided as input or learned during training. Following Sun et al. (2024), given the output graph representation $\mathbf{Z} \in \mathbb{R}^{n_o \times d}$ and the target-description matrix $\mathbf{C} \in \mathbb{R}^{c \times d}$, we unify different graph learning tasks by computing the inner-product similarity $\circ(\mathbf{Z}\mathbf{C}^T)$ as the final prediction. The function $\circ(\cdot)$ is implemented as `softmax` for classification and the identity mapping for regression. Here, n_o denotes the number of prediction targets, which equals n , $m + m_{neg}$, and 1 for node-, edge-, and graph-level tasks on a single graph, respectively. m_{neg} denotes the sampled negative edges which do not belong to $\mathcal{E}$.

Message Passing. For both GNNs and graph transformers, the fundamental graph operators can be formulated as

message passing (Veličković, 2022). Given a central node v_i , the message passing result $\mathbf{h}_{v_i}^{(l)}$ at the l -th layer $\mathbf{f}^{(l)}$ can be formulated as

$$\begin{aligned} \mathbf{h}_{v_i}^{(l)} &= [\mathbf{f}_{\Theta^{(l)}}(\mathbf{H}^{(l-1)}, \mathbf{A})]_{v_i} \\ &= \bigoplus_{v_j \in \mathcal{N}(v_i)} \left[\phi^{(l)}(\mathbf{h}_{v_i}^{(l-1)}), \psi^{(l)}(\mathbf{h}_{v_j}^{(l-1)}) \right], \end{aligned} \quad (1)$$

where $\mathbf{H}^{(l)} = (\mathbf{h}_{v_1}^{(l)}, \dots, \mathbf{h}_{v_n}^{(l)})^\top \in \mathbb{R}^{n \times d}$. $\mathbf{h}_{v_i}^{(0)} = \mathbf{x}_{v_i}$. $\phi^{(l)}$ and $\psi^{(l)}$ denote the feature-wise transformation, such as the linear mapping. $\oplus$ can be any aggregation function permutation invariant to the ordering of nodes, such as a graph convolution operator (Kipf & Welling, 2017; Hamilton et al., 2017; Veličković et al., 2018a) for message passing on the input graphs, while being a self-attention aggregator on the attention graph. $\Theta^{(l)}$ denotes the set of parameters for $\oplus$ at the l -th layer.

3.2. Recurrent Graph Model

Let $\mathbf{F} = \mathbf{f}_{\text{in}} \circ \mathbf{f}_{\Theta} \circ \dots \circ \mathbf{f}_{\Theta} \circ \mathbf{f}_{\text{out}}$ denote the whole recurrent graph model, where $\mathbf{f}_{\text{in}}$ and $\mathbf{f}_{\text{out}}$ are the input and output transformation, respectively. $\mathbf{f}_{\Theta}$ denotes the recurrent layer employed iteratively to refine the node representation. Given a graph $\mathcal{G}$, a target representation $\mathbf{C}$, and the corresponding ground-truth labels, the learning objective is to optimize the recurrent model $\mathbf{F}$ such that the inner-product relations between the output representations and the target representations are properly adjusted, thereby minimizing the prediction loss.

Recurrent Message Passing. The recurrent layer can be regarded as the discrete approximation of a continuous-time dynamical system. For time $t \in [0, T]$, let $\{\mathbf{H}^*(t)\}$ denote the target trajectory of the system governed by $\frac{d}{dt}\mathbf{H}^*(t) = \mathbf{v}(\mathbf{H}^*(t), \mathcal{C}, t)$, $\mathbf{H}^*(0) = \mathbf{H}^{(0)}$, where $\mathbf{v}(\cdot)$ is a continuous mapping that assigns the current representation $\mathbf{H}^*(t)$ a velocity matrix. $\mathcal{C}$ denotes the set of conditions such as the adjacent matrix $\mathbf{A}$. This process can be further discretized. Given a total recurrent step budget S , and step size $\Delta t = T/S$, the target discrete solution at the recurrent step s is $\mathbf{H}^{*(t_s)}$, where $t_s = s\Delta t$, $s = 0, 1, \dots, S$. Following the same form, Eq. 1 can be reformulated at the s -th step as

$$\begin{aligned} \mathbf{h}_{v_i}^{(t_s)} &= [\mathbf{f}_{\Theta}(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1})]_{v_i} \\ &= \bigoplus_{v_j \in \mathcal{N}(v_i)} \left[\phi(\mathbf{h}_{v_i}^{(t_{s-1})}), \psi(\mathbf{h}_{v_j}^{(t_{s-1})}) \right]. \end{aligned} \quad (2)$$

Different from traditional message passing, $\mathbf{f}_{\Theta}$ is employed recursively, with the parameter set Θ shared across the message-passing steps. A recurrent message-passing process aims to produce trajectories that stay close to the continuous reference $\{\mathbf{H}^*(t_s)\}_{s=0}^S$ at every step, which can be formally defined as uniform convergence.

Definition 3.1 (Uniform Convergence). Recall that for time $t \in [0, T]$, a step budget S , and step size $\Delta t = T/S$, the target discrete solution at the recurrent step s is $\mathbf{H}^{*(t_s)}$ with $t_s = s\Delta t$. Let $\{\mathbf{H}^{(t_s)}\}_{s=0}^S$ be the discrete trajectory generated by a recurrent layer. We say that $\mathbf{H}^{(t_s)}$ converges uniformly to $\mathbf{H}^{*(t_s)}$ if

$$\sup_{0 \leq s \leq S} \|\mathbf{H}^{(t_s)} - \mathbf{H}^{*(t_s)}\|_F \leq C\Delta t, \quad (3)$$

for some constant C independent of S . $\|\cdot\|_F$ denotes the Frobenius norm. The approximation error in uniform convergence vanishes uniformly over the entire time range. As $S \rightarrow \infty$, $C\Delta t \rightarrow 0$.

Step Information. The mapping in the target continuous system assigns a different velocity matrix at each step. This indicates that the recurrent process in Eq. 2 should depend on the step information $t_s = s\Delta t$ to achieve uniform convergence to specific target systems. Formally, we show that a recurrent layer without step information cannot converge to the target system uniformly, while a layer with step information can. For the target trajectory to be meaningful in the sense of convergence, we assume it is generated by a continuously differentiable Lipschitz function. This assumption is mild and non-restrictive, as it is a standard condition ensuring the well-posedness of trajectory dynamics.

Lemma 3.2 (Non-uniform Convergence without Step Information). *Let $\mathbf{f}_{\Theta}(\mathbf{H}^{(t_{s-1})}, \mathcal{C})$ be a recurrent layer that generates a discrete trajectory $\{\mathbf{H}^{(t_s)}\}_{s=0}^S$ without the step information $t_s = s\Delta t$. There exist a target trajectory $\{\mathbf{H}^*(t_s)\}_{s=0}^S$ generated by a continuously differentiable Lipschitz function and a constant $\varepsilon > 0$ such that, for sufficiently large S , $\sup_{0 \leq s \leq S} \|\mathbf{H}^{(t_s)} - \mathbf{H}^*(t_s)\|_F \geq \varepsilon$.*

Theorem 3.3 (Uniform Convergence with Step Information). *For any target trajectory $\{\mathbf{H}^*(t_s)\}_{s=0}^S$ generated by a continuously differentiable Lipschitz function, there exists a recurrent layer $\mathbf{H}^{(t_s)} = \mathbf{f}_{\Theta}(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1})$ such that $\mathbf{H}^{(t_s)}$ converges uniformly to $\mathbf{H}^*(t_s)$.*

Please refer to Appendix A for the proof. Based on Lemma 3.2 and Theorem 3.3, we can see that previous recurrent graph models fail to converge with different iteration budgets due to their miscapturing of the step information. This observation motivates the design of an adaptive recurrent graph model.

3.3. AdaR for Flexible Test-time Computing

In this section, we present our adaptive recurrent graph model, termed **AdaR**, following the recurrent message-passing paradigm. By explicitly encoding step information, AdaR can operate under varying iteration budgets without retraining or architectural modification, thereby enabling flexible test-time computing for different downstream tasks.

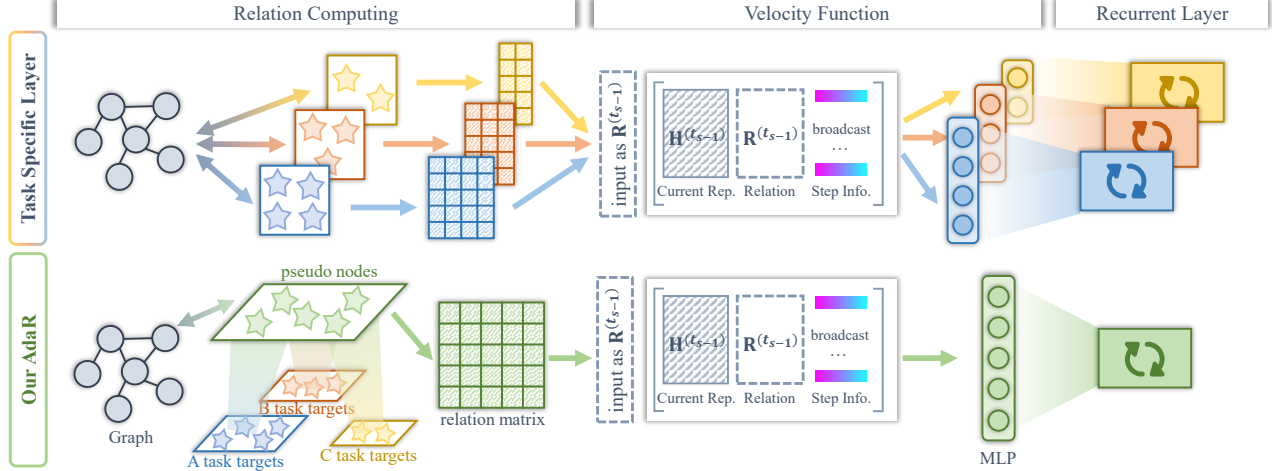


Figure 3. **Adaptability of AdaR.** To enable a target-aware recurrent process, AdaR introduces relations with targets to compute representation updates. However, the number of targets varies across tasks. To tackle this problem, a proxy-based relation computing strategy is designed, where a fixed number of pseudo nodes aggregate information from different numbers of targets, serving as target proxies for relation computing.

3.3.1. RECURRENT UPDATE WITH THE VELOCITY FUNCTION

Without loss of generality, a recurrent layer accessible to step information can be written in the residual form

$$\begin{aligned} \mathbf{H}^{(t_s)} &= \mathbf{f}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1}) \\ &= \mathbf{H}^{(t_{s-1})} + \mathbf{v}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1})\Delta t. \end{aligned} \quad (4)$$

Here, $\mathbf{v}_\Theta(\cdot)$ computes the velocity for nodes and updates $\mathbf{H}^{(t_{s-1})}$ to adjust their relation to the targets, ensuring the optimal output-target relation at the final step. To achieve this, $\mathbf{v}_\Theta(\cdot)$ should explicitly depend on

- the current representation $\mathbf{H}^{(t_{s-1})}$,
- the representation-target relations $\mathbf{R}^{(t_{s-1})}$, ensuring adaptive updates based on specific inputs and targets,
- the step information $\mathbf{S}_{s-1,\cdot}$, ensuring convergent approximation towards the targets,

where $\mathbf{S} \in \mathbb{R}^{s \times d}$ denotes the step matrix. The step velocity function $\mathbf{v}_\Theta(\cdot)$ can thus be formulated in a simple form

$$\begin{aligned} \mathbf{v}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1}) \\ = \text{NL} \left(\underbrace{\text{MP}(\mathbf{H}^{(t_{s-1})})}_{\text{current rep.}} \parallel \underbrace{\mathbf{R}^{(t_{s-1})}}_{\text{relation}} \parallel \underbrace{\mathbf{1}_n \mathbf{S}_{s-1,\cdot}^\top}_{\text{step info.}} \right), \end{aligned} \quad (5)$$

where $\mathcal{C} = \{\mathbf{A}, \mathbf{R}^{(t_s)}, \mathbf{S}_{s,\cdot}\}$, NL can be any non-linear transformation, such as a linear layer combined with ReLU. MP denotes message passing function, which is implemented following N^2 (Sun et al., 2024). Additional implementation

details are provided in Appendix C. $\parallel$ denotes the concatenation operation. Here, $\mathbf{1}_n \in \mathbb{R}^n$ is an all-one vector, such that $\mathbf{1}_n \mathbf{S}_{s-1,\cdot}^\top$ broadcasts the step embedding to all nodes. In the following context, we show how to prepare the last two features for computing the velocity.

3.3.2. ENCODING STEP INFORMATION

Given the step information $t_s = s\Delta t = sT/S$, two common design choices include fixing the unit time step Δt or fixing the total time T . Specifically, by fixing the total time (e.g., $T = 1$), different choices of the iteration budget S correspond to different discretizations of the same target trajectory, ensuring that the model is always trained to produce $\mathbf{H}^{(1)}$ as the final representation. In contrast, by fixing the unit time (i.e., $\Delta t = 1$), different iteration budgets S correspond to the same discretization scheme of different trajectories. Although this formulation simplifies the learning of $\mathbf{v}_\Theta$ to produce velocity within a fixed unit time, it may lead to out-of-distribution behavior when generalizing to unseen time and budgets. Based on the stability under different iteration budgets, AdaR fixes the total time $T = 1$. In Appendix D, we further provide theoretical and empirical analysis to validate the usage of a fixed total time rather than a fixed unit time step.

To map step scalars into the d -dimensional space, we propose to adopt the sinusoidal position encoding function (Vaswani et al., 2017), yielding the step matrix $\mathbf{S}$ as

$$\mathbf{S}_{s,j} = \begin{cases} \sin(\omega_k \cdot t_s), & \text{if } j = 2k \\ \cos(\omega_k \cdot t_s), & \text{if } j = 2k + 1, \end{cases} \quad (6)$$

where $\omega_k = \frac{1}{10000^{2k/d}}$. The sinusoidal encoding provides a continuous representation that is smoothly conditioned

on the step information t_s . Compared to learnable step embeddings, it avoids overfitting to a fixed number of iterations and naturally supports arbitrary step budgets without introducing additional parameters.

3.3.3. PROXY-BASED RELATION COMPUTING

Following the task formulation in Section 3.1, the representation-target relation can be formulated as $\mathbf{H}^{(t_{s-1})} \mathbf{C}^\top$. However, the number of targets varies across different graph learning tasks, making it infeasible for v_Θ to model node-wise velocities using a fixed set of parameters.

To address this issue, we introduce a set of pseudo nodes as proxies for targets (Fig. 3). These pseudo nodes aggregate information from the target representation via message passing and updates its own representations accordingly. As a result, the pseudo nodes can be regarded as abstractions of targets, serving as fixed-size proxies for relation computation. Let $\mathbf{P}^{(t_s)} \in \mathbb{R}^{n_c \times d}$ denote the pseudo-node representations, and $\mathbf{P}^{(t_0)}$ be a learnable initialization. The computation of the pseudo-node representations can be formulated as

$$\begin{aligned} \mathbf{P}^{(t_s)} &= \mathbf{P}^{(t_{s-1})} + v_\beta(\mathbf{P}^{(t_{s-1})}, \mathbf{S}_{s-1,\cdot}, t_{s-1}) \Delta t, \\ v_\beta(\cdot) &= \text{NL} \left(\underbrace{\text{AGG}(\mathbf{C})}_{\text{current rep.}} \parallel \underbrace{\mathbf{P}^{(t_{s-1})} \mathbf{P}^{(t_{s-1})^\top}}_{\text{relation}} \parallel \underbrace{\mathbf{1}_n \mathbf{S}_{s-1,\cdot}^\top}_{\text{step info.}} \right). \end{aligned} \quad (7)$$

The update process follows the velocity function in Eq. 5, with the aggregation function AGG implemented following $\mathbf{N}^2$ (Sun et al., 2024). Further details are presented in Appendix C.1. In consequence, the relation between current graph node representations and targets can be formulated as

$$\mathbf{R}^{(t_{s-1})} = \mathbf{H}^{(t_{s-1})} \mathbf{P}^{(t_s)^\top}. \quad (8)$$

In practice, to ensure the adaptability to different targets, we also recurrently update the target representation as $\mathbf{C}^{(t_s)}$. Similar to Eq. 5, the update of targets also depends on their current representation, step information, and their relation with graph nodes. For brevity and to avoid redundancy, we provide the details in Appendix C.3.

3.4. Training Strategy

Training a recurrent model requires supervision at every recurrent step. To achieve this, we propose to use the gradient of task loss with respect to the intermediate representations $\mathbf{H}^{(t_s)}$ as supervision signals, and align it with the predicted step displacement $-v_\Theta(\mathbf{H}^{(t_{s-1})}, \mathbf{C}, t_{s-1}) \Delta t$. Let $\mathcal{L}_{\text{task}}(\mathbf{Z})$ denote a task-dependent loss computed on the representation

Z. The overall optimization objective is defined as

$$\begin{aligned} \mathcal{L} &= \mathcal{L}_{\text{step}} + \mathcal{L}_{\text{full}} + \mathcal{L}_{\text{task}}(\mathbf{H}^{(t_s)}), \\ \mathcal{L}_{\text{step}} &= \frac{1}{S} \sum_{s=1}^S \left\| \underbrace{\nabla \mathbf{H}^{(t_{s-1})}}_{\text{target grad.}} - \underbrace{\left(-\Delta \mathbf{H}^{(t_s)} \right)}_{\text{pred. grad.}} \right\|_1, \\ \mathcal{L}_{\text{full}} &= \left\| \underbrace{\nabla \mathbf{H}^{(t_0)}}_{\text{target grad.}} - \underbrace{\left(\mathbf{H}^{(t_0)} - \mathbf{H}^{(t_S)} \right)}_{\text{pred. grad.}} \right\|_1, \\ \nabla \mathbf{H}^{(t_s)} &= \frac{\partial \mathcal{L}_{\text{task}}(\mathbf{H}^{(t_s)})}{\partial \mathbf{H}^{(t_s)}}, \\ \Delta \mathbf{H}^{(t_s)} &= v_\Theta(\mathbf{H}^{(t_{s-1})}, \mathbf{C}, t_{s-1}) \Delta t = \mathbf{H}^{(t_s)} - \mathbf{H}^{(t_{s-1})}. \end{aligned} \quad (9)$$

Here, $\mathcal{L}_{\text{step}}$ aligns the negative predicted step displacement $-\Delta \mathbf{H}^{(t_s)}$ with the task gradient $\nabla \mathbf{H}^{(t_{s-1})}$ at each recurrent step, encouraging every recurrent update to move the representation in a descent direction of $\mathcal{L}_{\text{task}}$. $\mathcal{L}_{\text{full}}$ enforces global consistency by matching the accumulated displacement of the entire recurrent process with the task gradient at the initial representation. $\mathcal{L}_{\text{task}}$ is directly optimized to ensure task performance and guide the learning process.

4. Experiment

In this section, we evaluate AdaR on transfer tasks under both inductive and transductive settings³, verifying its effectiveness in handling the varying receptive-field requirements at the graph level and the node level, respectively. Graph-level diversity stems from the varying optimal model depths across graphs (Fig. 1). For the node-level diversity, different nodes from the same graph may require different receptive fields. This is especially common in heterophilic graphs, where informative nodes can be located several hops away from the target node, while in homophilic graphs, useful nodes are often local.

4.1. Zero-shot Inductive Transfer

Experimental Setup. Both AdaR and baseline methods are jointly trained on four datasets (arXiv, bookhistory, amazon-ratings, and PubMed) (Chen et al., 2024b). After training, the models are evaluated in the zero-shot setting on downstream datasets (arxiv-year, WikiCS, SportsFit, Cora, CiteSeer, and DBLP) (Chen et al., 2024b). Statistics of the datasets are summarized in Tab. A4.

Four types of baseline models are incorporated, including GNN backbones developed for the supervised setting, self-supervised methods, an LLM, and graph foundation models. Please refer to Appendix E.2 for more details. The best

³<https://github.com/sunjss/AdaR>

Table 1. Evaluation Results in the Zero-shot Inductive Setting (measured by accuracy: %). † denotes the average number of parameters among the best model configurations on each dataset.

		TRAINABLE PARAMS	TOTAL PARAMS	ARXIV-YEAR	WIKICS	SPORTSFIT	CORA	CITESEER	DBLP
GNN BACKBONES	GCN	100%	0.4M	21.79 $\pm$ 2.66	27.21 $\pm$ 0.23	11.48 $\pm$ 0.16	37.91 $\pm$ 0.74	35.74 $\pm$ 2.42	51.69 $\pm$ 0.36
	GAT	100%	0.4M	18.96 $\pm$ 2.58	31.33 $\pm$ 0.19	11.95 $\pm$ 0.52	39.65 $\pm$ 0.71	36.52 $\pm$ 0.15	55.93 $\pm$ 1.59
	N ²	100%	0.4M	13.80 $\pm$ 0.49	22.39 $\pm$ 0.93	7.87 $\pm$ 0.97	29.30 $\pm$ 0.27	22.26 $\pm$ 1.05	36.19 $\pm$ 1.81
	GRAPHSAGE	100%	0.6M	18.41 $\pm$ 0.86	30.44 $\pm$ 0.97	12.20 $\pm$ 0.59	36.46 $\pm$ 0.14	33.23 $\pm$ 1.09	54.15 $\pm$ 1.72
	GCNII	100%	0.5M	25.59 $\pm$ 0.52	11.57 $\pm$ 1.46	13.33 $\pm$ 0.75	37.04 $\pm$ 1.48	23.82 $\pm$ 0.42	37.37 $\pm$ 0.79
SELF-SUPERVISED METHODS	DGI	100%	0.4M	OOM	8.26 $\pm$ 1.42	OOM	15.47 $\pm$ 1.07	26.81 $\pm$ 0.71	36.78 $\pm$ 1.99
	GRACE	100%	0.4M	OOM	7.26 $\pm$ 0.61	OOM	19.68 $\pm$ 2.76	25.39 $\pm$ 0.70	26.46 $\pm$ 1.58
FOUNDATION MODELS	VICUNA-7B	—	6.7B	—	29.31 $\pm$ 0.49	30.15 $\pm$ 0.16	14.39 $\pm$ 0.74	—	—
	LLAGA	0.02%	6.7B	21.20 $\pm$ 0.91	2.65 $\pm$ 0.13	5.45 $\pm$ 0.67	14.05 $\pm$ 0.57	15.62 $\pm$ 0.15	11.55 $\pm$ 0.89
	OFA	100%	5.1M	4.67 $\pm$ 0.14	36.89 $\pm$ 1.37	12.65 $\pm$ 1.41	39.45 $\pm$ 1.14	55.03 $\pm$ 2.09	51.01 $\pm$ 1.82
	GFT	100%	0.4M†	10.59 $\pm$ 1.61	6.77 $\pm$ 1.09	8.71 $\pm$ 1.49	25.15 $\pm$ 0.24	18.62 $\pm$ 0.45	31.75 $\pm$ 0.40
	MDGFM	100%	0.4M†	23.73 $\pm$ 1.15	22.87 $\pm$ 0.52	10.37 $\pm$ 0.72	10.66 $\pm$ 1.39	17.46 $\pm$ 0.87	20.84 $\pm$ 2.75
	RIEMANNGFM	100%	0.4M	1.01 $\pm$ 0.01	4.26 $\pm$ 0.49	3.83 $\pm$ 0.02	1.19 $\pm$ 0.05	11.76 $\pm$ 0.34	38.68 $\pm$ 0.37
	UNIGTE	0.54%	13.7B	—	34.91 $\pm$ 0.42	29.89 $\pm$ 0.15	21.29 $\pm$ 0.69	—	—
	ADAR (OURS)	100%	0.4M	36.15 $\pm$ 0.52	37.81 $\pm$ 0.94	31.56 $\pm$ 0.61	41.28 $\pm$ 0.82	59.39 $\pm$ 1.08	59.83 $\pm$ 1.02

Table 2. Evaluation Results in the Supervised Transductive Setting (measured by ROC-AUC except accuracy for amz-ratings: %).

(a) Heterophilic Graphs					(b) Homophilic Graphs				
	QUESTIONS	AMZ -RAINGS	TOLOKERS	MINE SWEEPER		COAUTHOR CS	COAUTHOR PHYSICS	AMZ PHOTO	AMZ COMPUTERS
GCN	76.09 $\pm$ 1.27	48.70 $\pm$ 0.63	83.64 $\pm$ 0.67	89.75 $\pm$ 0.52	GCN	92.92 $\pm$ 0.12	96.18 $\pm$ 0.07	92.70 $\pm$ 0.20	89.65 $\pm$ 0.52
GAT	77.43 $\pm$ 1.20	49.09 $\pm$ 0.63	83.70 $\pm$ 0.47	92.01 $\pm$ 0.68	GAT	93.61 $\pm$ 0.14	96.17 $\pm$ 0.08	93.87 $\pm$ 0.11	90.78 $\pm$ 0.17
GCNII	78.62 $\pm$ 0.21	48.44 $\pm$ 0.15	82.68 $\pm$ 1.24	92.94 $\pm$ 1.12	GCNII	95.03 $\pm$ 0.59	96.96 $\pm$ 0.13	94.84 $\pm$ 0.16	92.35 $\pm$ 0.09
GPRGNN	55.48 $\pm$ 0.91	44.88 $\pm$ 0.34	72.94 $\pm$ 0.97	86.24 $\pm$ 0.61	GPRGNN	95.13 $\pm$ 0.09	96.85 $\pm$ 0.08	94.49 $\pm$ 0.14	89.32 $\pm$ 0.29
H ₂ GCN	63.59 $\pm$ 1.46	36.47 $\pm$ 0.23	73.35 $\pm$ 1.01	89.71 $\pm$ 0.31	APNP	94.49 $\pm$ 0.07	96.54 $\pm$ 0.07	94.32 $\pm$ 0.14	90.18 $\pm$ 0.17
FAGCN	77.24 $\pm$ 1.26	44.12 $\pm$ 0.30	77.75 $\pm$ 1.05	88.17 $\pm$ 0.73	NAGPHORMER	95.75 $\pm$ 0.09	97.34 $\pm$ 0.03	95.49 $\pm$ 0.11	91.22 $\pm$ 0.14
GLOGNN	65.74 $\pm$ 1.19	36.89 $\pm$ 0.14	73.39 $\pm$ 1.17	51.08 $\pm$ 1.23	SAN	94.51 $\pm$ 0.15	OOM	94.86 $\pm$ 0.10	89.83 $\pm$ 0.16
GRAPHORMER	OOM	OOM	OOM	OOM	GRAPHORMER	OOM	OOM	92.74 $\pm$ 0.14	OOM
GRAPHGPS	OOM	OOM	84.70 $\pm$ 0.56	92.29 $\pm$ 0.61	GRAPHGPS	93.93 $\pm$ 0.12	OOM	95.06 $\pm$ 0.13	OOM
EXPHORMER	73.04 $\pm$ 0.58	49.37 $\pm$ 0.36	84.20 $\pm$ 0.22	90.42 $\pm$ 0.10	EXPHORMER	95.77 $\pm$ 0.15	97.16 $\pm$ 0.13	95.27 $\pm$ 0.42	91.59 $\pm$ 0.31
N ²	77.86 $\pm$ 0.56	49.70 $\pm$ 0.49	84.85 $\pm$ 0.53	93.97 $\pm$ 0.27	N ²	94.44 $\pm$ 0.45	97.56 $\pm$ 0.28	95.75 $\pm$ 0.34	92.51 $\pm$ 0.13
ADAR (OURS)	79.27 $\pm$ 0.51	49.98 $\pm$ 0.12	84.92 $\pm$ 0.65	94.08 $\pm$ 0.18	ADAR (OURS)	95.93 $\pm$ 0.23	97.89 $\pm$ 0.11	95.85 $\pm$ 0.25	93.38 $\pm$ 0.17

results among different backbones are reported for each self-supervised methods (Full results at Appendix E.3.1). The inference iteration steps are selected based on the node-target relation, with details reported in Section 4.3.

Performance. The comparison results between AdaR and baseline models are presented in Tab. 1. We can see that AdaR consistently achieves better performance across downstream tasks, which are collected from different domains and encompass both heterophilic and homophilic graphs. Compared to traditional GNN models, which do not follow the recurrent paradigm, AdaR achieves an average absolute improvement of 15.89%. Compared to N², which does not depend on step information and representation-target relations, AdaR achieves an average absolute improvement of 13.52%. In comparison with zero-shot graph foundation models, our flexible architecture empowers AdaR to employ a large iteration budget, revising the representation delicately without changing model parameters. As a result, AdaR achieves an average improvement of 21.99% over the compared graph foundation models. AdaR also requires fewer or comparable parameters to other baselines. All these results demonstrate the effectiveness of AdaR in tackling zero-shot inductive tasks, where enabling adaptive recurrent process and adjusting the number of iteration steps gives rise to better adaptability.

4.2. Supervised Transductive Transfer

Experimental Setup. Both AdaR and baselines are trained and evaluated on each dataset separately, including heterophilic (Platonov et al., 2023) and homophilic graphs (Shchur et al., 2019). Please refer to Appendix E.2 for more details. The best recurrent step of AdaR is selected based on the validation split.

Performance. Tab. 2 shows that AdaR with flexible architecture can also surpass baselines under the supervised setting, achieving consistent superior performance on both heterophilic and homophilic graphs. This indicates that even on the same graph, the nodes may require different sizes of receptive fields. Moreover, among AdaR, N², and graph transformers that adopt the global message-passing process, AdaR achieves an average absolute improvement of 1.33% than other models. This demonstrates that even with a global receptive field, the model still requires a flexible architecture to adjust the number of layers adaptively.

4.3. Model Analysis

Effectiveness of the Encoded Information. Based on our theoretical analysis, a recurrent model should explicitly depend on step information S at each iteration. To enable flex-

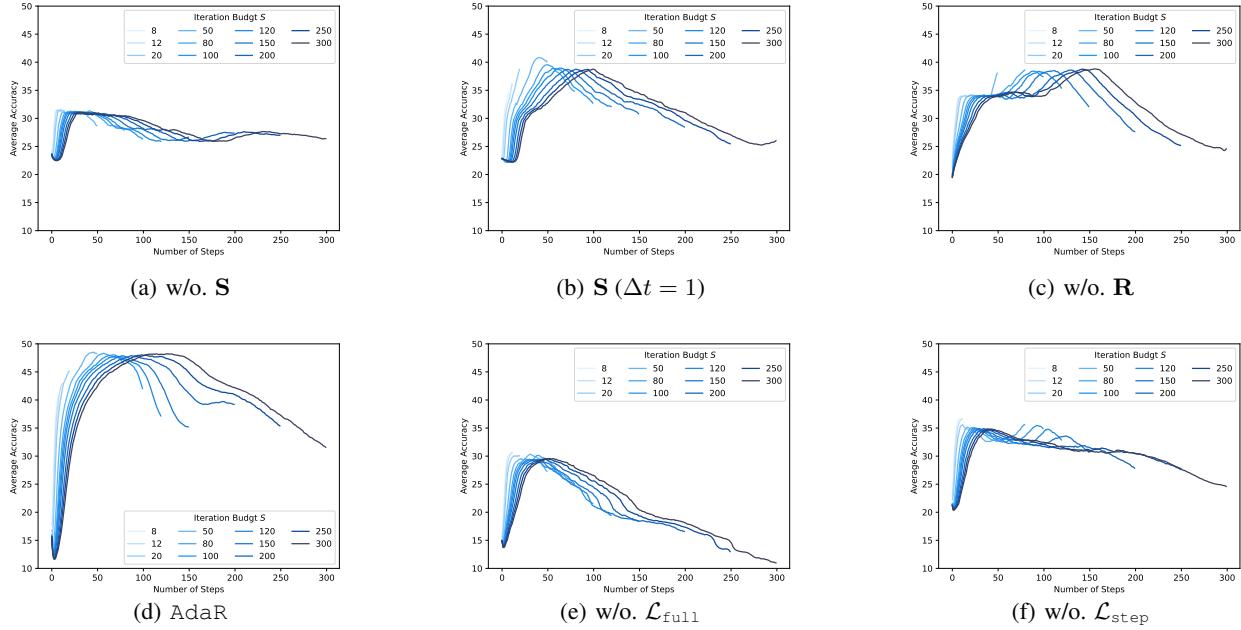


Figure 4. Test-time Computing under Different Design Choices. (a)-(c) studies the effectiveness of different encoded information, where w/o. $\mathbf{S}$ denotes no access to step information, $\mathbf{S}(\Delta t = 1)$ denotes encoding step information with a fixed unit step, w/o. $\mathbf{R}$ denotes not employing relations with targets. (d) presents the results of our AdaR. (e)-(f) studies the effectiveness of the supervision signals, where w/o. $\mathcal{L}_{\text{full}}/\mathcal{L}_{\text{step}}$ denotes not employing the corresponding optimization target.

ible inference under varying iteration budgets, AdaR adopts *relative* step encoding by fixing the total time $T = 1$, rather than using *absolute* time encoding that fixes the unit time $\Delta t = 1$. Moreover, to adapt to diverse inputs and targets, AdaR also incorporates the relations between the current node and target representations, denoted as $\mathbf{R}$. We conduct an ablation study to examine the effect of these design choices, as shown in Fig. 4(a)-4(c), and compare them with the full AdaR model in Fig. 4(d). Models are pre-trained on amazon-ratings, PubMed, arXiv, and BookHistory with an iteration budget of $S = 8$, and evaluated on arxiv-year, WikiCS, SportsFit, Cora, CiteSeer, and DBLP using iteration budgets in $\{8, 12, 20, 50, 80, 100, 120, 150, 200, 250, 300\}$. Results are averaged across all evaluation datasets. As shown in Fig. 4(a), removing step information $\mathbf{S}$ leads to the most severe performance degradation, as the model fails to approximate the target trajectory based on Theorem 3.2. Furthermore, compared to fixing the total time in AdaR, fixing the unit time $\Delta t = 1$ fails to generalize to unseen iteration budgets, where increasing the number of iterations results in deteriorating performance (Fig. 4(b)). Regarding the usage of relation information, AdaR, which encodes relations between nodes and targets, achieves better performance compared to the variant with no access to relations (Fig. 4(c)). These results collectively demonstrate the necessity of incorporating both normalized step information and target-aware relations for flexible recurrent inference.

Effectiveness of the Supervision Signals. To provide supervision at each recurrent step, AdaR introduces the gradient of the task loss with respect to the intermediate representations $\mathbf{H}^{(t_s)}$ as supervision signals, and matches it with the predicted updates of $\mathbf{H}^{(t_s)}$. Specifically, $\mathcal{L}_{\text{full}}$ enforces the overall recurrent update to align with the gradient computed at the initial representation, while $\mathcal{L}_{\text{step}}$ supervises the update at each individual step. We conduct ablation studies on these two optimization objectives in Fig. 4(e) and Fig. 4(f), following the same experimental setup as the step information ablation. As shown in the results, removing either $\mathcal{L}_{\text{full}}$ or $\mathcal{L}_{\text{step}}$ leads to performance degradation. Notably, removing $\mathcal{L}_{\text{full}}$ results in a more severe drop in performance (Fig. 4(e)). This is because $\mathcal{L}_{\text{full}}$ enforces global consistency of the recurrent updates across the entire process, whereas relying solely on $\mathcal{L}_{\text{step}}$ may cause local approximation errors to accumulate over iterations, ultimately harming the final representations. Regarding the ablation on $\mathcal{L}_{\text{step}}$, losing supervision at each recurrent step results in a more fluctuating inference process, where the curves in Fig. 4(f) exhibit multiple local maxima, indicating oscillatory updates across recurrent steps.

Iteration Exiting Strategy During empirical analysis, we observe that the model performance exhibits a unimodal pattern during the recurrent process, reaching a single maximum at an intermediate step (Fig. 4(d)). This suggests that

the predicted label distributions of nodes become progressively more aligned with the ground-truth distribution at early iterations, while later iterations lead to degraded or over-smoothed predictions. To characterize this behavior from a representation perspective, we analyze the average inner-product relation between node and target representations $\sum_{i,j} (\mathbf{H}^{(t_s)} \mathbf{C}^{(t_s)\top})_{i,j}$ at each recurrent step. Results in Fig. 5 show that the average value of the inner-product matrix also follows a unimodal pattern, whose maximum closely coincides with the peak model performance. This observation indicates that the average inner-product can serve as an effective surrogate for measuring the alignment between node representations and target representations, and thus provides a practical criterion for iteration exiting.

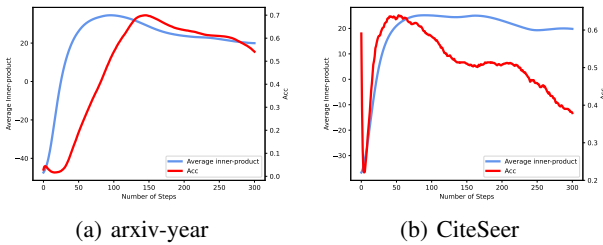


Figure 5. Iteration Exiting Strategy.

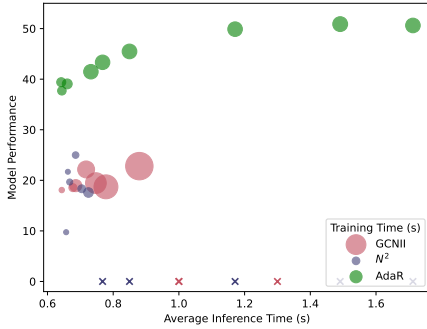


Figure 6. Accuray-Computing Curve. Bubble size denotes training time. ‘x’ denotes out-of-memory configurations.

Effectiveness under Constrained Resources. Different downstream tasks require various optimal receptive fields. To satisfy such requirements, graph nodes must be able to reach nodes at any distance via message passing. Previous works, such as GCNII (Chen et al., 2020), directly achieve this by implementing extremely deep graph models via residual mechanisms. However, this paradigm requires the same architectural configuration during training and inference, leading to longer training time and the out-of-memory issue. In contrast, by employing normalized step information, AdaR can be pre-trained with a small number of iteration budget and make inferences with significantly

Table 3. Cost Comparison.

		ADAR	N ²	LLaGA	UniGTE
TIME	TRAIN ($\times 10^3$)	1.59	0.54	80.11	260.44
(S)	TEST	4.59	0.74	2,170.34	2093.72
MEM.	TRAIN	9.87	5.94	29.82	30.74
(GB)	TEST	0.57	0.49	14.04	27.25

larger budgets. This empowers AdaR to achieve better performance with less training consumption. In practice, an ideal training budget is around 8, while the inference budget is around 300. To evaluate the effectiveness of our recurrent paradigm under constrained training resources, AdaR is compared with GCNII and N² regarding model performance and computing efficiency. Grid search is performed on GCNII and N² with the number of layers/iterations in {8, 20, 50, 100, 150, 200, 250, 300}. Results in Fig. 6 are averaged across different evaluation datasets, where AdaR demonstrates better performance with similar inference computing time and much less training time.

Cost Analysis. To demonstrate the absolute computing efficiency, AdaR is compared with N² and LLM-based methods LLaGA and UniGTE. The number of steps for N² is fixed as 8 during both training and inference. Compared to other foundation models, AdaR eliminates the dependence on LLMs to achieve adaptation to various downstream datasets, and therefore demands less memory overhead (Tab. 3). Notably, LLM-based methods transform the input graphs into subgraph sequences and process sequence mini-batches sequentially. As a result, these methods require significantly longer training and test time. This demonstrates the efficiency of AdaR in achieving promising zero-shot transfer performance. Please refer to Appendix E.3.2 for further time complexity analysis.

5. Conclusion

In this paper, we investigated how to reconcile the mismatch between the various receptive field requirements of graph data and the rigid design of pre-defined architectures. To address this problem, we proposed an adaptive recurrent graph model, termed AdaR, which provides a flexible architecture for adaptive test-time computing on graphs, tackling varying receptive field requirements without altering the model architecture. AdaR explicitly encodes normalized step information and representation–target relations into the recurrent updates, enabling adaptive recurrent process on different datasets. Gradient-based supervision signals are designed to guide representation updates throughout the recurrence in AdaR. Empirical results demonstrated that AdaR can be trained with a small iteration budget while being evaluated with substantially larger budgets, consistently outperforming baselines on both inductive and transductive tasks. Limitation discussion is provided in Appendix F.

Acknowledgement

This work was supported in part by the National Key R&D Program of China under Grant 2023YFC2508704, in part by the National Natural Science Foundation of China under grant number 62236008, in part by the Natural Science Foundation of Beijing under grant number L251082, and in part by Shandong Provincial Natural Science Foundation under project ZR2025ZD01. The authors would like to thank the anonymous reviewers for their helpful comments and suggestions that improved this manuscript.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Bo, D., Wang, X., Shi, C., and Shen, H. Beyond Low-frequency Information in Graph Convolutional Networks. In *AAAI Conference on Artificial Intelligence*, volume 35, pp. 3950–3957, virtual, 2021.
- Bresson, X. and Laurent, T. Residual Gated Graph ConvNets, 2018.
- Chen, J., Gao, K., Li, G., and He, K. NAGphormer: A Tokenized Graph Transformer for Node Classification in Large Graphs. In *International Conference for Learning Representations*, 2023.
- Chen, M., Wei, Z., Huang, Z., Ding, B., and Li, Y. Simple and Deep Graph Convolutional Networks. In *International Conference on Machine Learning*, pp. 1725–1735, virtual, 2020. PMLR.
- Chen, R., Zhao, T., Jaiswal, A. K., Shah, N., and Wang, Z. LLaGA: Large Language and Graph Assistant. In *International Conference on Machine Learning*, pp. 7809–7823. PMLR, 2024a.
- Chen, Z., Mao, H., Liu, J., Song, Y., Li, B., Jin, W., Fatemi, B., Tsitsulin, A., Perozzi, B., Liu, H., and Tang, J. Text-space Graph Foundation Models: Comprehensive Benchmarks and New Insights. In *Advances in Neural Information Processing Systems*, volume 37, pp. 7464–7492, December 2024b.
- Chien, E., Peng, J., Li, P., and Milenkovic, O. Adaptive Universal Generalized PageRank Graph Neural Network. In *International Conference on Learning Representations*, virtual, 2022.
- Fang, T., Zhang, Y., Yang, Y., Wang, C., and Chen, L. Universal Prompt Tuning for Graph Neural Networks. *Advances in Neural Information Processing Systems*, 36: 52464–52489, 2023.
- Gao, Z., Dong, D., Tan, C., Xia, J., Hu, B., and Li, S. Z. A Graph is Worth $\$K\$$ Words: Euclideanizing Graph using Pure Transformer. In *International Conference on Machine Learning*, June 2024.
- Gasteiger, J., Bojchevski, A., and Günnemann, S. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *International Conference on Learning Representations*, Addis Ababa, Ethiopia, 2018.
- Gui, A., Ye, J., and Xiao, H. G-Adapter: Towards structure-aware parameter-efficient transfer learning for graph transformer networks. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence*, volume 38, pp. 12226–12234. AAAI Press, 2024.
- Hamilton, W. L., Ying, R., and Leskovec, J. Inductive representation learning on large graphs. In *International Conference on Neural Information Processing Systems*, pp. 1025–1035, Red Hook, USA, 2017. Curran Associates Inc.
- Hu, Z., Dong, Y., Wang, K., Chang, K.-W., and Sun, Y. GPT-GNN: Generative Pre-Training of Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1857–1867, New York, NY, USA, 2020. Association for Computing Machinery.
- Hu, Z., Li, Y., Chen, Z., Wang, J., Liu, H., Lee, K., and Ding, K. Let’s Ask GNN: Empowering Large Language Model for Graph In-Context Learning. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 1396–1409, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- Huang, Q., Ren, H., Chen, P., Kržmanc, G., Zeng, D., Liang, P., and Leskovec, J. PRODIGY: Enabling In-context Learning Over Graphs. In *Conference on Neural Information Processing Systems*, 2023.
- Huang, X., Han, K., Yang, Y., Bao, D., Tao, Q., Chai, Z., and Zhu, Q. Can GNN be Good Adapter for LLMs? In *Proceedings of the ACM Web Conference 2024*, pp. 893–904, New York, NY, USA, 2024. Association for Computing Machinery.
- Kingma, D. P. and Ba, J. Adam: A Method for Stochastic Optimization. In *International Conference for Learning Representations*, San Diego, USA, 2015.

- Kipf, T. N. and Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*, Toulon, France, 2017.
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., Dollár, P., and Girshick, R. Segment Anything, 2023.
- Kong, L., Feng, J., Liu, H., Huang, C., Huang, J., Chen, Y., and Zhang, M. GOfA: A Generative One-For-All Model for Joint Graph Language Modeling. In *The Thirteenth International Conference on Learning Representations*, October 2024.
- Kreuzer, D., Beaini, D., Hamilton, W., Létourneau, V., and Tossou, P. Rethinking Graph Transformers with Spectral Attention. In *Advances in Neural Information Processing Systems*, volume 34, pp. 21618–21629. Curran Associates, Inc., 2021.
- Leskovec, J. and Krevl, A. SNAP Datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>, 2014.
- Li, S., Han, X., and Bai, J. AdapterGNN: Parameter-Efficient Fine-Tuning Improves Generalization in GNNs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 13600–13608, 2024a.
- Li, X., Zhu, R., Cheng, Y., Shan, C., Luo, S., Li, D., and Qian, W. Finding Global Homophily in Graph Neural Networks When Meeting Heterophily. In *International Conference on Machine Learning*, volume 162, pp. 13242–13256, Baltimore, Maryland, USA, 2022. PMLR.
- Li, Y., Tarlow, D., Brockschmidt, M., and Zemel, R. Gated Graph Sequence Neural Networks. In *International Conference on Learning Representations*, pp. 1–20, 2017.
- Li, Y., Wang, P., Li, Z., Yu, J. X., and Li, J. ZeroG: Investigating Cross-dataset Zero-shot Transferability in Graphs. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1725–1735, New York, NY, USA, 2024b. Association for Computing Machinery.
- Liu, H., Feng, J., Kong, L., Liang, N., Tao, D., Chen, Y., and Zhang, M. One For All: Towards Training One Graph Model For All Classification Tasks. In *International Conference on Learning Representations*, 2023a.
- Liu, Z., Yu, X., Fang, Y., and Zhang, X. GraphPrompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks. In *Proceedings of the ACM Web Conference 2023*, pp. 417–428, New York, NY, USA, 2023b. Association for Computing Machinery.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., and Avila, R. e. a. GPT-4 Technical Report, 2024.
- Pan, B., Zhang, Z., Zhang, Y., Hu, Y., and Zhao, L. Distilling Large Language Models for Text-Attributed Graph Learning. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pp. 1836–1845, New York, NY, USA, 2024. Association for Computing Machinery.
- Platonov, O., Kuznedelev, D., Diskin, M., Babenko, A., and Prokhorenkova, L. A critical look at the evaluation of GNNs under heterophily: Are we really making progress? In *The Eleventh International Conference on Learning Representations*, Kigali, Rwanda, 2023.
- Qiu, J., Chen, Q., Dong, Y., Zhang, J., Yang, H., Ding, M., Wang, K., and Tang, J. GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training. In *ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1150–1160, New York, NY, USA, 2020. Association for Computing Machinery.
- Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., and Beaini, D. Recipe for a General, Powerful, Scalable Graph Transformer. In *Advances in Neural Information Processing Systems*, volume 35, pp. 14501–14515, New Orleans, USA, 2022.
- Ruiz, L., Gama, F., and Ribeiro, A. Gated Graph Recurrent Neural Networks. *IEEE Transactions on Signal Processing*, 68:6303–6318, 2020. ISSN 1941-0476.
- Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., and Monfardini, G. The Graph Neural Network Model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.
- Selsam, D., Lamm, M., B\”{u}nz, B., Liang, P., de Moura, L., and Dill, D. L. Learning a SAT Solver from Single-Bit Supervision. In *International Conference on Learning Representations*, 2018.
- Shchur, O., Mumme, M., Bojchevski, A., and Günnemann, S. Pitfalls of Graph Neural Network Evaluation. *arXiv:1811.05868 [cs, stat]*, 2019.
- Shirzad, H., Vellingker, A., Venkatachalam, B., Sutherland, D. J., and Sinop, A. K. Expformer: Sparse Transformers for Graphs. In *International Conference on Machine Learning*, volume 202, Honolulu, USA, 2023. PMLR.
- Sun, J., Wang, S., Han, X., Xue, Z., and Huang, Q. All in a row: Compressed convolution networks for graphs. In *International Conference on Machine Learning*, volume 202, pp. 33061–33076, Honolulu, USA, 2023a. PMLR.

- Sun, J., Yang, C., Ji, X., Huang, Q., and Wang, S. Towards Dynamic Message Passing on Graphs. In *Conference on Neural Information Processing Systems*, December 2024.
- Sun, J., Chang, W., Yang, C., Huang, Q., and Wang, S. Relieving the Over-Aggregating Effect in Graph Transformers. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a.
- Sun, L., Huang, Z., Zhou, S., Wan, Q., Peng, H., and Yu, P. S. RiemannGFM: Learning a Graph Foundation Model from Structural Geometry. In *THE WEB CONFERENCE 2025*, January 2025b.
- Sun, M., Zhou, K., He, X., Wang, Y., and Wang, X. GPPT: Graph Pre-training and Prompt Tuning to Generalize Graph Neural Networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1717–1727, New York, NY, USA, 2022. Association for Computing Machinery.
- Sun, X., Cheng, H., Li, J., Liu, B., and Guan, J. All in One: Multi-Task Prompting for Graph Neural Networks. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 2120–2131, New York, NY, USA, 2023b. Association for Computing Machinery.
- Tan, Z., Guo, R., Ding, K., and Liu, H. Virtual Node Tuning for Few-shot Node Classification. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 2177–2188, New York, NY, USA, 2023. Association for Computing Machinery.
- Tang, J., Yang, Y., Wei, W., Shi, L., Su, L., Cheng, S., Yin, D., and Huang, C. GraphGPT: Graph Instruction Tuning for Large Language Models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 491–500, New York, NY, USA, 2024. Association for Computing Machinery.
- Tian, Y., Song, H., Wang, Z., Wang, H., Hu, Z., Wang, F., Chawla, N. V., and Xu, P. Graph Neural Prompting with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19080–19088, 2024.
- Van der Maaten, L. and Hinton, G. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *International Conference on Neural Information Processing Systems*, pp. 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc.
- Veličković, P. Message passing all the way up. In *International Conference on Learning Representation Workshop on Geometrical and Topological Representation Learning*, 2022.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph Attention Networks. In *International Conference on Learning Representations*, Vancouver, Canada, 2018a.
- Veličković, P., Fedus, W., Hamilton, W. L., Liò, P., Bengio, Y., and Hjelm, R. D. Deep Graph Infomax. In *International Conference on Learning Representations*, 2018b.
- Wang, D., Zuo, Y., Lu, G., and Wu, J. UniGTE: Unified Graph–Text Encoding for Zero-Shot Generalization across Graph Tasks and Domains. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a.
- Wang, S., Wang, B., Shen, Z., Deng, B., and Kang, Z. Multi-Domain Graph Foundation Models: Robust Knowledge Transfer via Topology Alignment. In *Forty-Second International Conference on Machine Learning*, 2025b.
- Wang, Z., Zhang, Z., Chawla, N. V., Zhang, C., and Ye, Y. GFT: Graph Foundation Model with Transferable Tree Vocabulary. In *Advances in Neural Information Processing Systems*, volume 37, pp. 107403–107443, 2024. doi: 10.52202/079017-3412.
- Xia, L., Kao, B., and Huang, C. OpenGraph: Towards Open Graph Foundation Models. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 2365–2379, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., and Liu, T.-Y. Do Transformers Really Perform Bad for Graph Representation? In *Advances in Neural Information Processing Systems*, volume 34, pp. 28877–28888. Curran Associates, Inc., 2021.
- Yu, X., Fang, Y., Liu, Z., Wu, Y., Wen, Z., Bo, J., Zhang, X., and Hoi, S. C. H. A Survey of Few-Shot Learning on Graphs: From Meta-Learning to Pre-Training and Prompt Learning, 2024a.
- Yu, X., Zhou, C., Fang, Y., and Zhang, X. MultiGPrompt for Multi-Task Pre-Training and Prompting on Graphs. In *Proceedings of the ACM Web Conference 2024*, pp. 515–526, New York, NY, USA, 2024b. Association for Computing Machinery.
- Zhang, M., Sun, M., Wang, P., Fan, S., Mo, Y., Xu, X., Liu, H., Yang, C., and Shi, C. GraphTranslator: Aligning Graph Model to Large Language Model for Open-ended Tasks. In *The Web Conference 2024*, May 2024.

- Zhang, W., Zhu, Y., Chen, M., Geng, Y., Huang, Y., Xu, Y., Song, W., and Chen, H. Structure Pretraining and Prompt Tuning for Knowledge Graph Transfer. In *Proceedings of the ACM Web Conference*, pp. 2581–2590, New York, NY, USA, 2023. Association for Computing Machinery.
- Zhao, H., Chen, A., Sun, X., Cheng, H., and Li, J. All in One and One for All: A Simple yet Effective Method towards Cross-domain Graph Pretraining. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 4443–4454, New York, NY, USA, 2024. Association for Computing Machinery.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, December 2023.
- Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., and Koutra, D. Beyond homophily in graph neural networks: Current limitations and effective designs. In *International Conference on Neural Information Processing Systems*, pp. 7793–7804, Red Hook, NY, USA, 2020a. Curran Associates Inc.
- Zhu, Y., Xu, Y., Yu, F., Liu, Q., Wu, S., and Wang, L. Deep Graph Contrastive Representation Learning. In *International Conference on Machine Learning Workshop on Graph Representation Learning and Beyond*, 2020b.
- Zhu, Y., Shi, H., Wang, X., Liu, Y., Wang, Y., Peng, B., Hong, C., and Tang, S. GraphCLIP: Enhancing Transferability in Graph Foundation Models for Text-Attributed Graphs. In *The Web Conference*, 2025.

A. Proof

A.1. Lemma 3.2

Lemma 3.2 (Non-uniform Convergence without Step Information) *Let $\mathbf{f}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C})$ be a recurrent layer that generates a discrete trajectory $\{\mathbf{H}^{(t_s)}\}_{s=0}^S$ without the step information $t_s = s\Delta t$. There exist a target trajectory $\{\mathbf{H}^*(t_s)\}_{s=0}^S$ generated by a continuously differentiable Lipschitz function and a constant $\varepsilon > 0$ such that, for sufficiently large S , $\sup_{0 \leq s \leq S} \|\mathbf{H}^{(t_s)} - \mathbf{H}^*(t_s)\|_F \geq \varepsilon$.*

Proof. Let the target trajectory $\{\mathbf{H}^*(t_s)\}_{s=0}^S$ be governed by the ordinary differential equation $\frac{d}{dt_s} \mathbf{H}^*(t_s) = \mathbf{v}(\mathbf{H}^*(t_s), \mathcal{C}, t_s)$. Applying a first-order Taylor expansion of $\mathbf{H}^*(t_s)$ at t_0 , we obtain

$$\mathbf{H}^*(t_s) = \mathbf{H}^*(t_0) + \mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0)(t_s - t_0) + O(t_s^2).$$

In particular, for $s = 1$, $\mathbf{H}^*(t_1) = \mathbf{H}^*(t_0) + \mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0)\Delta t + O(\Delta t^2)$.

Let the recurrent layer produce $\mathbf{H}^{(t_1)} = \mathbf{H}^{(t_0)} + \delta$, where δ is independent of Δt since the layer does not observe the step information $t_s = s\Delta t$. Assume, for contradiction, that the recurrent layer achieves uniform convergence without the step information. Then there exists a constant $C > 0$, such that for any target trajectory $\{\mathbf{H}^*(t_s)\}_{s=0}^S$,

$$\|\mathbf{H}^{(t_s)} - \mathbf{H}^*(t_s)\|_F \leq C\Delta t, \quad s \in \{0, \dots, S\}.$$

For $s = 1$, this implies

$$\begin{aligned} C\Delta t &\geq \|\mathbf{H}^{(t_1)} - \mathbf{H}^*(t_1)\| \\ &= \|\delta - \mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0)\Delta t - O(\Delta t^2)\| \\ C\Delta t + \|O(\Delta t^2)\| &\geq \|\delta - \mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0)\Delta t - O(\Delta t^2)\| + \|O(\Delta t^2)\|. \end{aligned}$$

By the triangle inequality, there exists a constant $K_1 > 0$ such that

$$\begin{aligned} \|\delta - \mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0)\Delta t\| &\leq C\Delta t + K_1\Delta t^2 \\ \left\| \frac{\delta}{\Delta t} - \mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0) \right\| &\leq C + K_1\Delta t. \end{aligned}$$

Let $S \rightarrow \infty$, so that $\Delta t = \frac{T}{S} \rightarrow 0$. Since δ is independent of Δt , we have $\frac{\delta}{\Delta t} \rightarrow \infty$ unless $\delta = 0$ and $\mathbf{H}^{(t_1)} = \mathbf{H}^{(t_0)}$. Hence, a step-independent recurrent layer can achieve uniform convergence only for target trajectories with $\|\mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0)\| = 0$. This contradicts the assumption. Therefore, for any target trajectory satisfying $\|\mathbf{v}(\mathbf{H}^*(t_0), \mathcal{C}, t_0)\| \neq 0$, there exists $\varepsilon > 0$ such that, for sufficiently large S , $\sup_{0 \leq s \leq S} \|\mathbf{H}^{(t_s)} - \mathbf{H}^*(t_s)\|_F \geq \varepsilon$. $\square$

A.2. Theorem 3.3

Lemma A.1 (Necessity of Step Information). *Let $\mathcal{H} \subset \mathbb{R}^m$ and $\mathcal{A} \subset \mathbb{R}^{n \times n}$ be compact and $\mathbf{v}(\mathbf{H}, \mathcal{C}, t) : \mathcal{H} \times \mathcal{A} \times [0, T] \rightarrow \mathbb{R}^d$ be continuous. There exists a family of functions $\{\mathbf{v}_\Theta\}$ that uniformly approximates $\mathbf{v}$ on $\mathcal{H} \times \mathcal{A} \times [0, T]$, if and only if the layer input includes a variable that uniquely determines the step information t .*

Proof.

Sufficiency. By the universal approximation theorem, for any $\varepsilon > 0$, there exists a parameterized function $\mathbf{v}_\Theta : \mathcal{H} \times \mathcal{A} \times [0, T] \rightarrow \mathbb{R}^d$ (e.g., a multi-layer perceptron), such that

$$\sup_{(\mathbf{H}, \mathcal{C}, t) \in \mathcal{H} \times \mathcal{A} \times [0, T]} \|\mathbf{v}(\mathbf{H}, \mathcal{C}, t) - \mathbf{v}_\Theta(\mathbf{H}, \mathcal{C}, t)\| < \varepsilon. \quad (\text{A10})$$

This establishes sufficiency.

Necessity. Since $\mathbf{v} : \mathcal{H} \times \mathcal{A} \times [0, T] \rightarrow \mathbb{R}^d$ is non-autonomous, there exist $t_1 \neq t_2$, $\mathbf{H}, \mathcal{C}$, such that $\mathbf{v}(\mathbf{H}, \mathcal{C}, t_1) \neq \mathbf{v}(\mathbf{H}, \mathcal{C}, t_2)$. Then the necessity of the lemma can be formulated as that there does not exist a function $\mathbf{g} : \mathcal{H} \times \mathcal{A} \rightarrow \mathbb{R}^d$ can uniformly approximate $\mathbf{v}$ on $\mathcal{H} \times \mathcal{A} \times [0, T]$.

Assume, for contradiction, that for arbitrarily small $\varepsilon > 0$, there exists a function g satisfying $\sup_{\mathbf{H}, \mathcal{C}, t} \|\mathbf{v}(\mathbf{H}, \mathcal{C}, t) - g(\mathbf{H}, \mathcal{C})\| \leq \varepsilon$. In particular, for input $\mathbf{H}, \mathcal{C}$, and $t_1 \neq t_2$, we have

$$\|\mathbf{v}(\mathbf{H}, \mathcal{C}, t_1) - g(\mathbf{H}, \mathcal{C})\| \leq \varepsilon, \quad \|\mathbf{v}(\mathbf{H}, \mathcal{C}, t_2) - g(\mathbf{H}, \mathcal{C})\| \leq \varepsilon. \quad (\text{A11})$$

By the triangle inequality,

$$\|\mathbf{v}(\mathbf{H}, \mathcal{C}, t_1) - \mathbf{v}(\mathbf{H}, \mathcal{C}, t_2)\| \leq \|\mathbf{v}(\mathbf{H}, \mathcal{C}, t_1) - g(\mathbf{H}, \mathcal{C})\| + \|\mathbf{v}(\mathbf{H}, \mathcal{C}, t_2) - g(\mathbf{H}, \mathcal{C})\| \leq 2\varepsilon. \quad (\text{A12})$$

Since ε can be chosen arbitrarily small, this implies $\mathbf{v}(\mathbf{H}, \mathcal{C}, t_1) = \mathbf{v}(\mathbf{H}, \mathcal{C}, t_2)$, which contradicts the non-autonomy. Therefore, uniform approximation is impossible without the step information. $\square$

Theorem 3.3 (Uniform Convergence with Step Information) *For any target trajectory $\{\mathbf{H}^{*(t_s)}\}_{s=0}^S$ generated by a continuously differentiable Lipschitz function, there exists a recurrent layer $\mathbf{H}^{(t_s)} = \mathbf{f}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1})$ such that $\mathbf{H}^{(t_s)}$ converges uniformly to $\mathbf{H}^{*(t_s)}$.*

Proof. Without loss of generality, a recurrent layer accessible to step information can be written in the residual form $\mathbf{f}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1}) = \mathbf{H}^{(t_{s-1})} + \mathbf{v}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1})\Delta t$. Based on Lemma A.1, for any $\varepsilon > 0$ there exists a mapping function $\mathbf{v}_\Theta$ such that $\sup_{(\mathbf{H}, \mathcal{C}, t) \in \mathcal{H} \times \mathcal{A} \times [0, T]} \|\mathbf{v}(\mathbf{H}, \mathcal{C}, t) - \mathbf{v}_\Theta(\mathbf{H}, \mathcal{C}, t)\| < \varepsilon$.

Let the target trajectory $\{\mathbf{H}^{*(t_s)}\}_{s=0}^S$ satisfy the Lipschitz condition with respect to $\mathbf{H}^{*(t_s)}$, i.e., $\|\mathbf{v}(\mathbf{H}_1^{*(t_s)}, \mathcal{C}, t) - \mathbf{v}(\mathbf{H}_2^{*(t_s)}, \mathcal{C}, t)\| \leq \mathcal{L}_\mathbf{H} \|\mathbf{H}_1^{*(t_s)} - \mathbf{H}_2^{*(t_s)}\|$. By Taylor expansion, $\mathbf{H}^{*(t_{s+1})} = \mathbf{H}^{*(t_s)} + \mathbf{v}(\mathbf{H}^{*(t_s)}, \mathcal{C}, t_s)\Delta t + O(\Delta t^2)$. Let $\mathbf{e}_s$ denote the error $\mathbf{H}^{(t_s)} - \mathbf{H}^{*(t_s)}$. Then

$$\begin{aligned} \mathbf{e}_{s+1} &= \mathbf{H}^{(t_{s+1})} - \mathbf{H}^{*(t_{s+1})} \\ &= \mathbf{H}^{(t_s)} + \mathbf{v}_\Theta(\mathbf{H}^{(t_s)}, \mathcal{C}, t_s)\Delta t - \mathbf{H}^{*(t_s)} - \mathbf{v}(\mathbf{H}^{*(t_s)}, \mathcal{C}, t_s)\Delta t - O(\Delta t^2) \\ &= \mathbf{e}_s + \Delta t \left[\mathbf{v}_\Theta(\mathbf{H}^{(t_s)}, \mathcal{C}, t_s) - \mathbf{v}(\mathbf{H}^{*(t_s)}, \mathcal{C}, t_s) \right] - O(\Delta t^2). \end{aligned} \quad (\text{A13})$$

Decomposing the difference term yields

$$\begin{aligned} \mathbf{e}_{s+1} &= \mathbf{e}_s + \Delta t \left[\mathbf{v}_\Theta(\mathbf{H}^{(t_s)}, \mathcal{C}, t_s) - \mathbf{v}(\mathbf{H}^{*(t_s)}, \mathcal{C}, t_s) \right] - O(\Delta t^2) \\ &= \mathbf{e}_s + \Delta t \left[\underbrace{\mathbf{v}_\Theta(\mathbf{H}^{(t_s)}, \mathcal{C}, t_s) - \mathbf{v}(\mathbf{H}^{(t_s)}, \mathcal{C}, t_s)}_{\text{Universal Approximation Error}} + \underbrace{\mathbf{v}(\mathbf{H}^{(t_s)}, \mathcal{C}, t_s) - \mathbf{v}(\mathbf{H}^{*(t_s)}, \mathcal{C}, t_s)}_{\text{Lipschitz Condition}} \right] - O(\Delta t^2) \end{aligned} \quad (\text{A14})$$

By uniform approximation, the Lipschitz condition, and the triangle inequality, we obtain

$$\|\mathbf{e}_{s+1}\| \leq (1 + \mathcal{L}_\mathbf{H}\Delta t)\|\mathbf{e}_s\| + \varepsilon\Delta t + K_1\Delta t^2, \quad (\text{A15})$$

for some constant $K_1 > 0$.

Applying induction and using $\mathbf{e}_0 = 0$, we have

$$\begin{aligned} \|\mathbf{e}_{s+1}\| &\leq (1 + \mathcal{L}_\mathbf{H}\Delta t)^{s+1} \underbrace{\|\mathbf{e}_0\|}_{=0} + \sum_{i=0}^s (1 + \mathcal{L}_\mathbf{H}\Delta t)^i (\varepsilon\Delta t + K_1\Delta t^2) \\ &= \frac{(1 + \mathcal{L}_\mathbf{H}\Delta t)^{s+1} - 1}{\mathcal{L}_\mathbf{H}\Delta t} (\varepsilon\Delta t + K_1\Delta t^2) \\ &= \frac{\exp[s \ln(1 + \mathcal{L}_\mathbf{H}\Delta t)] - 1}{\mathcal{L}_\mathbf{H}} (\varepsilon + K_1\Delta t) \\ &\leq \frac{\exp[s\mathcal{L}_\mathbf{H}\Delta t] - 1}{\mathcal{L}_\mathbf{H}} (\varepsilon + K_1\Delta t) \\ &= \frac{\exp[t_s\mathcal{L}_\mathbf{H}] - 1}{\mathcal{L}_\mathbf{H}} (\varepsilon + K_1\Delta t). \end{aligned} \quad (\text{A16})$$

Since $t_s \leq T$, it follows that $\sup_s \|\mathbf{e}_s\| \leq \frac{\exp(T\mathcal{L}_\mathbf{H}) - 1}{\mathcal{L}_\mathbf{H}} (\varepsilon + K_1\Delta t)$. When $\varepsilon \rightarrow 0$, the error bound decreases to $\frac{\exp(T\mathcal{L}_\mathbf{H}) - 1}{\mathcal{L}_\mathbf{H}} K_1\Delta t = C\Delta t$, which proves the uniform convergence. $\square$

B. Pseudo Nodes in Adaptive Graph Models

To unify the varying number of targets across different tasks, AdaR adopts pseudo nodes to aggregate information from the target representation and update its own representations accordingly. As a result, these pseudo nodes can be regarded as abstractions of targets, serving as fixed-size proxies for relation computation during the recurrent process. The concept of pseudo nodes has emerged in many adaptive graph models (Liu et al., 2023a; Wang et al., 2024; Zhao et al., 2024). However, AdaR introduces pseudo nodes for a different purpose via a different implementation strategy. Regarding purposes, pseudo nodes in previous works are to readout the learned representations (Liu et al., 2023a; Wang et al., 2024) or to enhance the cross-graph communication (Zhao et al., 2024), yet AdaR employs pseudo nodes to unify different targets. Regarding implementations, pseudo nodes in AdaR are shared parameters for all graphs. Conversely, pseudo nodes in previous works either cannot be jointly optimized with the overall framework or fail to transfer across tasks (Liu et al., 2023a; Wang et al., 2024; Zhao et al., 2024).

C. Implementation of MP

We implement MP based on the global and local message passing in $\mathbf{N}^2$ (Sun et al., 2024) with minor modification, including replacing the path integral function with a combination of inner product and `softmax`, replacing the update function with a velocity function in both Eq. A19 and Eq. A26.

C.1. Global Message Passing and Implementation of AGG

Given two sets of node elements, such as graph nodes, pseudo nodes, and targets, one set of elements can perform global message passing via the other set of elements. For example, when the number of graph nodes is extremely large, graph nodes can perform global message passing with a small number of pseudo nodes as surrogates to ensure linear complexity. Let $\mathbf{X}_{\text{in}}$ denote the input representation of the global message-passing process, $\mathbf{X}_{\text{sur}}$ denote the surrogate representation, and $\mathbf{X}_{\text{cond}}$ denote a certain condition representation, such as target representations. The global message-passing process at the s -th step can be formulated as

$$\text{(Diffuse)} \quad \mathbf{G} = \psi(\mathbf{X}_{\text{sur}} \mathbf{X}_{\text{in}}^{\top}) \text{NL}(\mathbf{X}_{\text{in}}), \quad (\text{A17})$$

$$\text{(Refine)} \quad \hat{\mathbf{G}} = \psi(\mathbf{X}_{\text{sur}} \mathbf{X}_{\text{sur}}^{\top}) \mathbf{G}, \quad (\text{A18})$$

$$\mathbf{X}_{\text{sur}} \leftarrow \mathbf{X}_{\text{sur}} + \text{NL}([\hat{\mathbf{G}} \| \mathbf{1}_n \mathbf{S}_{s,\cdot}^{\top} \| \mathbf{X}_{\text{sur}} \mathbf{X}_{\text{cond}}^{\top}) \Delta t, \quad (\text{A19})$$

$$\text{(Collect)} \quad \mathbf{M}^{\text{glob}} = \psi(\mathbf{X}_{\text{in}} \mathbf{X}_{\text{sur}}^{\top}) \text{NL}(\hat{\mathbf{G}}), \quad (\text{A20})$$

where ψ constructs edge weights for message passing between different node elements, such as graph nodes, pseudo nodes, and targets. In practice, ψ is implemented as a combination of inner product and `softmax`. Eq. A19 formulates the global feature refinement at the surrogate level, where $\text{NL}([\hat{\mathbf{G}} \| \mathbf{1}_n \mathbf{S}_{s,\cdot}^{\top} \| \mathbf{X}_{\text{sur}} \mathbf{X}_{\text{cond}}^{\top})$ follows the same form in Eq. 5, considering the current representation, the step information, its relation with certain conditions. Eq. A20 formulates the global message-passing outputs, where $\mathbf{M}^{\text{glob}}$ encodes global information.

This process can be summarized as “input→surrogate→input”. For simplicity, we compile Eq. A17-A20 as $\mathbf{M}^{\text{glob}}, \mathbf{X}_{\text{sur}} \leftarrow \text{GlobMP}(\mathbf{X}_{\text{in}}, \mathbf{X}_{\text{sur}}, \mathbf{X}_{\text{cond}})$. Eq. A17 and Eq. A18 formulate the aggregation function $\text{AGG}(\cdot)$ in Eq. 7.

C.2. Local Message Passing

Topology-coupled message passing is employed to encode the local structures. The resulted local message-passing process for graph node v can be formulated as

$$\mathbf{m}_{v_i}^{\text{local}} = \frac{1}{|\mathcal{N}(v_i)| + 1} \left[\mathbf{m}_{v_i} + \sum_{v' \in \mathcal{N}(v_i)} \text{NL}(\mathbf{m}_{v_j}) \right]. \quad (\text{A21})$$

Through local message passing, graph nodes aggregate messages from their adjacent nodes. Eq. A21 can be compiled as $\mathbf{M}^{\text{local}} = \text{LocalMP}(\mathbf{M}, \mathbf{A})$ for all the graph nodes.

C.3. Velocity Computing

Based on the formulation from $\mathbf{N}^2$, we now specify the computing process in AdaR. Specifically, given the current representations of graph nodes and targets, AdaR first performs `GlobMP` to update pseudo-node representations and collect global features. In practice, we also update the target representations recurrently at this stage, depending on their current representation $\mathbf{C}^{(t_{s-1})}$, step information $\mathbf{S}$, and their relation with graph nodes. Therefore, similar to the computation of node-target relations, this process also requires pseudo nodes as surrogates. Let $\mathbf{P}^{(p,t_s)} \in \mathbb{R}^{n_p \times d}$ denote the pseudo-node representations, where $p \in \{n, c\}$ indicates the associated element type (graph nodes or targets), n_p is the number of pseudo nodes, and $\mathbf{P}^{(p,0)}$ is a learnable initialization. Given graph node representation $\mathbf{H}^{(t_{s-1})}$, target representation $\mathbf{C}^{(t_{s-1})}$, and pseudo-node representation $\mathbf{P}^{(p,t_{s-1})}$, $p \in \{c, n\}$ at the s -th recursive step, the velocity computing process can be formulated as

$$\text{(Graph-node Abs.)} \quad \mathbf{M}_{\text{pn}}^{\text{glob}(t_s)}, \mathbf{P}^{(n,t_s)} = \text{GlobMP} \left(\mathbf{H}^{(t_{s-1})}, \mathbf{P}^{(n,t_{s-1})}, \mathbf{P}^{(n,t_{s-1})} \right), \quad (\text{A22})$$

$$\text{(Target Abs.)} \quad \mathbf{P}^{(c,t_s)} = \text{GlobMP} \left(\mathbf{C}^{(t_{s-1})}, \mathbf{P}^{(c,t_{s-1})}, \mathbf{P}^{(c,t_{s-1})} \right), \quad (\text{A23})$$

$$\text{(Target Rep. Update)} \quad \mathbf{M}_{\text{cn}}^{\text{glob}(t_s)}, \mathbf{C}^{(t_s)} = \text{GlobMP} \left(\mathbf{H}^{(t_{s-1})}, \mathbf{C}^{(t_{s-1})}, \mathbf{P}^{(n,t_s)} \right), \quad (\text{A24})$$

$$\text{(Local MP.)} \quad \mathbf{M}^{\text{local}(t_s)} = \text{LocalMP} \left[\left(\mathbf{M}_{\text{pn}}^{\text{glob}(t_s)} \parallel \mathbf{M}_{\text{cn}}^{\text{glob}(t_s)} \parallel \mathbf{H}^{(t_{s-1})} \right), \mathbf{A} \right], \quad (\text{A25})$$

$$\text{(Node Rep. Update)} \quad \Delta \mathbf{H}^{(t_s)} = \text{NL}([\mathbf{M}^{\text{local}(t_s)} \parallel \mathbf{1}_n \mathbf{S}_{s-1}^\top \parallel \mathbf{H}^{(t_{s-1})} \mathbf{P}^{(c,t_s)\top}]), \quad (\text{A26})$$

where $\mathbf{M}_{\text{pn}}^{\text{glob}(t_s)}$, $\mathbf{M}_{\text{cn}}^{\text{glob}(t_s)}$ denote the global features of graph nodes, extracted via pseudo nodes and targets, respectively. Eq. A22 formulates the graph-node abstraction process with pseudo nodes (`Graph-node Abs.`). It takes graph-node representation $\mathbf{H}^{(t_{s-1})}$ as inputs $\mathbf{X}_{\text{in}}$, pseudo-node representation $\mathbf{P}^{(n,t_{s-1})}$ as the global message-passing surrogate $\mathbf{X}_{\text{sur}}$ and condition $\mathbf{X}_{\text{cond}}$, updating the pseudo-node representations to serve as an abstraction of graph nodes. Similarly, Eq. A23 formulates the target abstraction process with pseudo nodes (`Target Abs.`). It takes target representation $\mathbf{C}^{(t_{s-1})}$ as inputs $\mathbf{X}_{\text{sur}}$, pseudo-node representation $\mathbf{P}^{(c,t_{s-1})}$ as the global message-passing surrogate $\mathbf{X}_{\text{sur}}$ and condition $\mathbf{X}_{\text{cond}}$, updating the pseudo-node representations to serve as an abstraction of targets. Eq. A24 updates the target representations conditioned on the graph node abstraction $\mathbf{P}^{(n,t_s)}$ (`Target Rep Update`). Eq. A26 follows Eq. 5 to compute the velocity.

Compiling Eq. A22-A25 gives rise to the message passing function $\text{MP}(\cdot)$, while compiling Eq. A22-A26 yields the velocity function $\text{v}_\Theta(\mathbf{H}^{(t_{s-1})}, \mathcal{C}, t_{s-1})$.

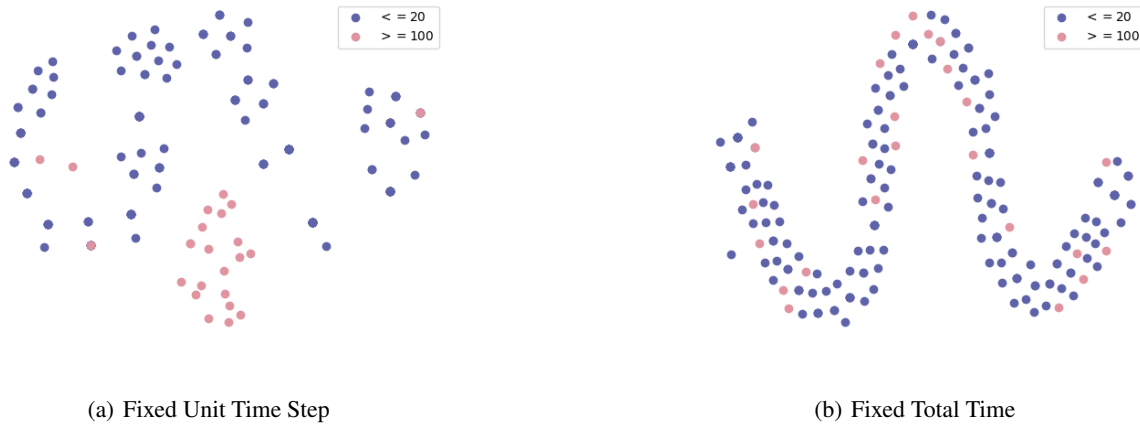


Figure A7. **Comparison with Different Step Information.** ‘ ≤ 20 ’ denotes iteration budgets sampled from $[2, 20]$, ‘ ≥ 100 ’ denotes iteration budgets sampled from $[100, 300]$.

Table A4. Dataset Statistics.

DATASET	TASK TYPE	# NODES	# EDGES	# CLASSES
ARXIV	INDUCTIVE	169,343	1,166,243	40
BOOKHISTORY	INDUCTIVE	41,551	358,574	12
PUBMED	INDUCTIVE	19,717	44,338	3
ARXIV-YEAR	INDUCTIVE	169,343	1,166,243	5
CORA	INDUCTIVE	2,708	10,556	7
CITESEER	INDUCTIVE	3,186	8,450	6
DBLP	INDUCTIVE	14,376	431,326	4
SPORTSFIT	INDUCTIVE	173,055	1,773,500	13
WIKICS	INDUCTIVE	11,701	431,726	10
AMAZON-RATINGS	INDUCTIVE/TRANSDUCTIVE	24,492	93,050	5
QUESTIONS	TRANSDUCTIVE	48,921	245,778	2
TOLOKERS	TRANSDUCTIVE	11,758	119,043	2
MINESWEEPER	TRANSDUCTIVE	10,000	163,788	2
AMAZONCOMPUTER	TRANSDUCTIVE	13,381	495,924	10
AMAZONPHOTO	TRANSDUCTIVE	7,487	153,540	8
COAUTHORPHYSICS	TRANSDUCTIVE	18,333	519,000	15
COAUTHORCS	TRANSDUCTIVE	34,493	39,402	5

D. Analysis on Step Information Encoding

Given the step information $t_s = s\Delta t = sT/S$, two common design choices include fixing the unit time step Δt or fixing the total time T . To ensure generalizability to different iteration budgets, AdaR fixes the total time (*e.g.*, $T = 1$). Specifically, different choices of the iteration budget S correspond to different discretizations of the same target trajectory, ensuring that the model is always trained to produce $\mathbf{H}^{(1)}$ as the final representation. To validate this specific implementation choice, we provide further analysis.

Theoretical Analysis. Based on the implementation of the sinusoidal position encoding function, we have step information embedded as $[\sin(w_k \cdot t_s), \cos(w_k \cdot t_s)]_k$. Let the largest iteration budget during training be T_0 . Without fixing the total time T , inference may involve $T \gg T_0$, leading to phase shifts $w_k \cdot (t_s - T_0)$ proportional to T and large embedding deviations. In contrast, fixing the total time with $T = 1$ rescales t_s to $[0, 1]$. Therefore, the maximum shift is bounded by the constant w_k independent of T , implying smaller embedding shifts.

Empirical Analysis. To empirically validate our encoding strategy, we select the iteration budget from two distinct ranges, *i.e.* smaller budgets $[2, 20]$ and larger budgets $[100, 300]$. The step indices are embedded with sinusoidal encoding and visualized with t-SNE (Van der Maaten & Hinton, 2008). Results in Fig. A7(a) show that when embedded with a fixed unit time step, embeddings from the smaller and larger budget ranges form clearly separated clusters with a distinct boundary, indicating a significant distribution shift between the two groups. In contrast, Fig. A7(b) shows that using a fixed total time yields a more mixed distribution, where embeddings from both ranges lie on a shared low-dimensional manifold, effectively alleviating the distribution shift. These results demonstrate the effectiveness of using a fixed total time rather than a fixed unit time step for step information encoding.

E. Experimental Details

E.1. Datasets

For inductive transfer tasks, we adopt the textual attribute datasets (Chen et al., 2024b), including (arXiv, bookhistory, amazon-ratings, and PubMed) for training and (arxiv-year, WikiCS, SportsFit, Cora, CiteSeer, and DBLP) for evaluation. arXiv, arxiv-year, PubMed, Cora, CiteSeer, and DBLP are citation graphs. Bookhistory, amazon-ratings, and SportsFit are economic-commercial graphs, while WikiCS is collected from Wikipedia.

For transductive tasks, we adopt both heterophilic datasets (questions, amazon-ratings, tolokera, and minesweeper) (Platonov et al., 2023) and homophilic datasets (CoauthorCS, CoauthorPhysics, AmazonPhoto, and AmazonComputers) (Shchur et al., 2019). Specifically, CoauthorCS and CoauthorPhysics are derived from the Microsoft Academic Graph and model academic collaboration networks. AmazonComputers and AmazonPhoto are product co-purchase networks collected from Amazon. Questions is built from the Yandex Q&A platform and represents user interaction graphs collected over a one-year period

Table A5. Full Results for Self-supervised Methods (measured by accuracy: %).

	BACKBONES	ARXIV-YEAR	WIKICS	SPORTSFIT	CORA	CITESEER	DBLP
DGI	GAT	OOM	16.14	9.13	17.88	OOM	5.40
	GCN	OOM	14.73	15.28	35.88	OOM	8.26
	GRAPHSAGE	OOM	26.81	15.47	36.78	OOM	2.82
GRACE	GAT	OOM	16.77	19.68	23.29	OOM	3.84
	GCN	OOM	18.18	10.15	26.46	OOM	5.06
	GRAPHSAGE	OOM	25.39	4.59	19.86	OOM	7.26
ADAR (OURS)		36.15	37.81	31.56	41.28	59.39	59.83

(September 2021–August 2022). Amazon-ratings is based on the Amazon co-purchasing network provided by the SNAP dataset collection (Leskovec & Krevl, 2014). Tolokers represents crowdsourcing participation data from the Toloka platform. Minesweeper is a synthetic grid graph of size 100×100 , where nodes correspond to grid cells.

E.2. Experimental Setup

In both inductive and transductive experiments, we follow the standard splits as in the original papers for all the benchmarks. We adopt Adam (Kingma & Ba, 2015) as optimizer and set weight decay as 1×10^{-6} . For inductive experiments, the training epoch is set as 10,000, the learning rate is set to 1×10^{-4} , the training iteration budget is set as 8, and the test-time iteration budget is set as 300. For transductive experiments, the training epoch is set as 1,000, the learning rate is set to 1×10^{-3} , the training iteration budget is selected from $\{2, 4, 8\}$, the test-time iteration budget is selected from $\{8, 20, 50, 100, 150, 200, 250, 300\}$ and the dropout probability is selected from $\{0, 0.1, 0.2, 0.3\}$. Models are selected based on the average performance on training datasets for the zero-shot setting, while based on validation sets for the transductive setting.

For the comparison under the zero-shot inductive transfer setting, we incorporate four types of baseline models, including supervised GNNs (GCN (Kipf & Welling, 2017), GAT (Veličković et al., 2018a), GCNII (Chen et al., 2020), GraphSAGE (Hamilton et al., 2017), and the recurrent graph model N^2 (Sun et al., 2024)), self-supervised methods (DGI (Veličković et al., 2018b), GRACE (Zhu et al., 2020b)), a large language model Vicuna-7B (Zheng et al., 2023), and graph foundation models (OFA (Liu et al., 2023a), LLaGA (Chen et al., 2024a), GFT (Wang et al., 2024), MDGFM (Wang et al., 2025b), RiemannGFM (Sun et al., 2025b), and UniGTE (Wang et al., 2025a)).

For the comparison under the supervised transductive setting, we incorporate both GNNs (GCN (Kipf & Welling, 2017), GAT (Veličković et al., 2018a), APPNP (Gasteiger et al., 2018), GCNII (Chen et al., 2020), H_2 GCN (Zhu et al., 2020a), FAGCN (Bo et al., 2021), GPRGNN (Chien et al., 2022), GloGNN (Li et al., 2022), and N^2 (Sun et al., 2024)) and graph transformers (Graphormer (Ying et al., 2021), GraphGPS (Rampásek et al., 2022), NAGphormer (Chen et al., 2023), SAN (Kreuzer et al., 2021), and Exphormer (Shirzad et al., 2023)).

Except for the reproducing experiments conducted on NVIDIA A100 for Vicuna-7B (Zheng et al., 2023), OFA (Liu et al., 2023a), LLaGA (Chen et al., 2024a), RiemannGFM (Sun et al., 2025b), UniGTE (Wang et al., 2025a), and cost comparison, all the experiments are conducted on a single NVIDIA GeForce RTX 4090.

For the cost comparison, models are pre-trained on arXiv, bookhistory, amazon-ratings, and PubMed, evaluated on WikiCS, SportsFit, and Cora.

E.3. Additional Results

E.3.1. FULL RESULTS FOR ZERO-SHOT INDUCTIVE TRANSFER

For baselines, we incorporate four types of baseline models, including supervised GNNs, self-supervised methods, a large language model Vicuna-7B, and zero-shot graph foundation models. The self-supervised methods are implemented with GCN, GAT, and GraphSAGE. The best results among the three backbones are reported for each method in the main context. Here, we present the full results in Tab. A5, where AdaR consistently outperforms self-supervised methods with different backbones.

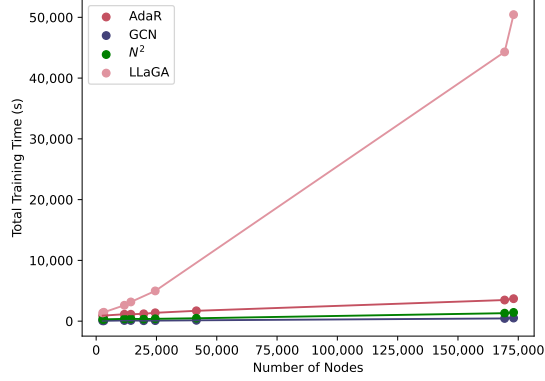


Figure A8. Training Time Regarding Different Numbers of Nodes.

E.3.2. COMPLEXITY ANALYSIS

Similar to classical GNNs such as GCN and GAT, AdaR requires a computing complexity of $O(n + m)$, where n and m denote the number of nodes and edges, respectively. In contrast, LLM-based models such as LLaGA often require a computing complexity of $\frac{O(k^2) \cdot n}{B}$, where B and k denote the batch size and the number of nodes in the mini-batch, respectively. Due to the quadratic time complexity of LLMs, these models can only operate on mini-batches and compute multiple batches in a sequential manner. Empirically, we conduct complexity analysis on real-world datasets with varying scales. Baselines include classic GNN model GCN, recurrent model N^2 , adaptive recurrent model AdaR, and LLM-based model LLaGA. Results in Fig. A8 show the linear time complexity of AdaR, comparable to other GNNs. Conversely, LLaGA exhibits significantly longer training time due to the requirement of batch processing.

F. Limitation

This paper proposes an adaptive recurrent graph model, AdaR, enabling flexible test-time computation on graphs. In the current implementation, AdaR still operates under a pre-defined maximum iteration budget S . In practice, we set $S = 300$ for zero-shot evaluation, while allowing early termination based on the average inner-product similarity between node representations and targets. Despite this practical flexibility, determining an input-adaptive criterion that automatically infers the optimal iteration budget remains an open problem, which we leave for future work.